Reading Without Diacritics: How Open LLMs Lose, and Partly Recover, Vietnamese Understanding When Tone Marks Disappear

Anonymous Author(s)
Affiliation withheld for review

***Abstract*—Vietnamese is often typed without diacritics, yet diacritics mark tone and vowel quality: unaccented *ban* may be *bạn* (friend), *bán* (sell) or *bàn* (table). We measure how much open large language models (LLMs) lose on such input, what predicts the loss, and what recovers it. We evaluate 19 instruction-tuned models (1.5–14B parameters, eight families) zero-shot on five Vietnamese tasks under seven noise conditions, with a validity gate fixed before any noisy result was examined. Removing all diacritics costs 41–67% of a model's above-chance accuracy, depending on the task, and the loss grows monotonically with the share of stripped syllables. The loss is unchanged across four prompt templates, including English instructions (Spearman ≥ 0.95 between model rankings), and shrinks by 0.086 per doubling of parameters, a trend that holds when three 14B models are added. Comparing models on the same noisy item, the one that finds the stripped input more surprising errs more often, also among models that share a tokenizer. Telling the model that the text may lack diacritics recovers none of the loss and re-estimating the calibration prior on the noisy inputs recovers 7%, whereas restoring diacritics with an off-the-shelf restorer before prompting recovers 64%. On 226 comments that their authors typed without diacritics, models score no better than on synthetically stripped comments, yet restoration does not help (+0.01 balanced accuracy, 95% CI $[-0.02, 0.04]$). We release the noise suite, notebooks with built-in checks, and per-item predictions.**

***Index Terms*—Vietnamese, diacritics, robustness, large language models, tokenization, diacritic restoration**

I. Introduction

Vietnamese uses diacritics for six tones and for several vowel qualities, so one unaccented string can stand for many words. Readers resolve the ambiguity from context, and many people type without diacritics in chat, comments and search queries: on keyboards without a Vietnamese input method, on mobile devices, or out of habit [1], [2]. A model deployed on Vietnamese user text will see such input.

Earlier work showed that removing diacritics hurts fine-tuned Vietnamese encoders on social-media tasks [2] and zero-shot Vietnamese classifiers [3], where it was the most damaging of the perturbations tested. Practitioners, however, increasingly use open generative LLMs with a prompt and no fine-tuning, and for them four questions remain open:

1) **How much** is lost, and does it depend on the prompt, on the amount of noise and on model size?
2) **Who** is robust: larger models, or models adapted to Vietnamese?
3) **Why**: because unaccented text is tokenized differently, or because the model finds it surprising?
4) **What helps**: telling the model, or restoring the diacritics first? And does a fix measured on synthetic noise carry over to text people actually typed without diacritics?

We answer them with 19 open LLMs of 1.5–14B parameters on five tasks. Our contributions are:

- A controlled measurement with full and partial diacritic removal, teen code and their mix, scored with single-token answers and a validity gate fixed in advance (§III–IV). The loss is monotone in the dose and does not depend on the prompt.
- Evidence that the loss shrinks steadily with model size up to 14B, while Vietnamese-adapted models are not more robust (§V).
- A same-item analysis in which the increase in model surprise predicts errors even when tokenization is identical, together with a robust negative association of broken syllables that we report as unexplained (§VI).
- A head-to-head comparison of mitigations on the same model–task pairs (a prompt hint, a noise-estimated calibration prior and automatic restoration), and a test on naturally unaccented comments (§VII).

II. Related Work

Vietnamese diacritics and robustness. ViSoBERT [2] removes 25–100% of the diacritics from five Vietnamese social-media datasets and reports that all fine-tuned encoders, its own included, degrade, with smaller losses at lower removal rates. It attributes PhoBERT's larger loss to a tokenizer that cannot encode unaccented text. Ha et al. [3] test four zero-shot "decision models" (an API model and three of 287–328M parameters) on 13 classification settings. Diacritic removal lowers mean macro-F1 by 9.5–15.5 points, against at most 1.1 for teen code and 4.5 for typos. Their LLM numbers are taken from earlier clean-input experiments [4] and are not re-run under noise. Neither study evaluates generative LLMs under diacritic removal, varies the prompt or model size, or compares mitigations. A recent benchmark tests LLMs on Vietnamese dialects [5] but not on diacritics. We extend this line to zero-shot generative LLMs and add size, prompt, mechanism and mitigation analyses. We also qualify the observation that teen code is harmless: in [3], teen code edits 2.1% of characters

TABLE I
TASKS AND TEST ITEMS (BALANCED PER CLASS).

Task	Type	Classes	Items
UIT-VSFC [12]	student-feedback sentiment	3	450
UIT-VSMEC [13]	social-media emotion	7	245
ViHSD [14]	social-media hate speech	3	450
XNLI-vi [15]	natural language inference	3	450
ViMMRC [16]	multiple-choice reading	4	240
Total			1,835

against 18.3% for diacritic removal, and at an equal dose the two do similar damage (§IV-C).

Diacritics and tokenization. Tokenizers trained mostly on English split many other scripts and diacritized text into more tokens, which raises cost and can lower performance [6], [7]. Gorman and Pinter [8] argue that NLP pipelines should not strip or inconsistently encode diacritics. We test whether changes in tokenization explain the loss once item and model are held fixed.

Diacritic restoration. Restoration is a well-studied sequence-labelling problem; Vietnamese is among the hardest languages for it [1]. Le-Hong [9] applies diacritic generation to hate speech detection on Vietnamese social networks. We evaluate an off-the-shelf restorer as a preprocessing defence for zero-shot LLMs and compare it with simply telling the model.

Synthetic versus natural noise. Robustness to synthetic noise need not transfer to natural noise [10]. We therefore also test restoration on comments that their authors typed without diacritics. Our protocol follows perturbation-based behavioural testing [11].

III. EXPERIMENTAL SETUP

A. Tasks and items

Table I lists the five public tasks [12]–[16]. Test items come from the test split. Dev items, used only to choose the answer format and the calibration prior, come from the disjoint validation split. We apply NFC normalisation and remove duplicates, items with conflicting labels, items under three syllables and items with no diacritic. We then sample a balanced number of items per class, 1,835 test items in total. Noise is applied to the input fields only; the instruction and the answer options stay clean.

B. Noise conditions

- `clean`: the original text.
- `partial25/50/75`: an exact share of the syllables that carry diacritics is stripped, drawn with three random seeds.
- `strip`: all diacritics are removed (NFD decomposition, combining marks dropped, đ→d).
- `teencode`: a hand-written substitution table (e.g. không→ko, được→dc); it changes 6–12% of words.
- `mixed`: teen code, then 50% stripping.

TABLE II
MODELS. VI: ADAPTED TO VIETNAMESE OR SOUTHEAST ASIAN LANGUAGES.

Family	Sizes (B)	Tokenizer	VI
Qwen2.5-Instruct	1.5, 3, 7, 14	Qwen (shared)	
Qwen3	1.7, 4, 8, 14	Qwen (shared)	
SeaLLMs-v3-Chat	1.5, 7	Qwen (shared)	✓
VinaLLaMA-chat	2.7, 7	VinaLLaMA	✓
BLOOMZ	1.7, 3, 7.1	BLOOM	
Phi-3.5-mini / phi-4	3.8 / 14	Phi	
Mistral-Instruct-v0.3	7	Mistral	
Llama-3.2-Instruct	3	Llama-3	

Automatic checks stop the run if `strip` leaves a diacritic, if `partial` changes anything besides diacritics, or if the noise is not deterministic.

C. Models

We evaluate 19 instruction-tuned models (Table II). Several families come in more than one size with the same tokenizer, which allows size comparisons with the tokenizer held fixed. Qwen2.5, Qwen3 and SeaLLMs-v3 share one tokenizer vocabulary, which gives a natural control for tokenization. Four models are adapted to Vietnamese or Southeast Asian languages by continued training: SeaLLMs-v3, released before Qwen2.5 and built on the Qwen2 architecture, and VinaLLaMA, built on Llama-2 [17]. The 14B models ran on one A100 80 GB GPU and the others on one L4.

D. Prompting and scoring

Each model receives its documented chat format; for Qwen3 the thinking mode is disabled (`enable_thinking=False`). Options are presented as letters, and the answer is prefilled with "Đáp án:" ("Answer:") after the template's assistant header. For every model we check that each option letter is a single, distinct token. We read the probability of each letter from one forward pass, which also yields the summed negative log-likelihood (NLL) of the input fields. We normalise it by the number of syllables of the *clean* input, so that ΔNLL does not depend on how many tokens a tokenizer produces (§VI). The main predictions use batch calibration [18] with a prior estimated on clean dev items. Raw predictions are kept and lead to the same conclusions (§IV-D).

E. Validity gate

Before examining any noisy result, we fixed three conditions that a (model, task) pair must meet to be analysed:

- median probability mass on the option letters ≥ 0.5;
- agreement between a free greedy answer and the scored answer ≥ 0.7, on 40 clean and 40 stripped items;
- clean accuracy $\geq$ chance $+ 0.10$, with the lower bound of its 95% CI above chance.

For the 16 models up to 8B, 74 of 80 pairs pass. The six exclusions all have clean accuracy near chance: five on ViHSD and BLOOMZ-1.7B on XNLI. The three 14B models pass on all five tasks, so 89 pairs are analysed in total.

TABLE III
MEAN NORMALISED DROP OVER VALID MODELS UP TO 8B, AND THE EFFECT OF AUTOMATIC RESTORATION (§VII). WORD ACC.: SHARE OF WORDS THE RESTORER GETS RIGHT.

	`strip`		`restored`		
Task	d [95% CI]	n	Word acc.	d	Recovery [95% CI]
VSFC	0.67 [0.61, 0.74]	16	92.5%	0.12	0.77 [0.68, 0.85]
VSMEC	0.66 [0.55, 0.76]	16	76.2%	0.23	0.64 [0.50, 0.78]
XNLI	0.56 [0.45, 0.64]	15	92.0%	0.10	0.73 [0.62, 0.87]
ViHSD	0.53 [0.44, 0.63]	11	75.1%	0.24	0.52 [0.32, 0.66]
ViMMRC	0.41 [0.36, 0.46]	16	81.2%	0.15	0.55 [0.40, 0.67]

F. Metrics and statistics

The *normalised drop* under condition c is

$$d_c = \frac{\text{acc}_{\text{clean}} - \text{acc}_c}{\text{acc}_{\text{clean}} - \text{chance}}, \tag{1}$$

which is 0 for no loss and 1 for a fall to chance. For restoration we report the *recovery*

$$r = \frac{\text{acc}_{\text{restored}} - \text{acc}_{\text{strip}}}{\text{acc}_{\text{clean}} - \text{acc}_{\text{strip}}}, \tag{2}$$

computed when the strip loss exceeds 0.05. CIs are 95% paired bootstrap intervals over items ($B = 2000$, with the same resamples for every condition). Within-family tests use Holm correction. Regressions use fixed effects as stated, with standard errors clustered by item (§VI) or by model (§V).

IV. How Much Is Lost?

A. Full removal

Table III shows the normalised drop under `strip`. A typical model loses between two fifths and two thirds of its above-chance accuracy. Reading comprehension (ViMMRC) loses least. Its long passages may give more context for resolving ambiguous words, but we have not tested this.

B. Dose-response

Accuracy falls monotonically with the share of stripped syllables in 86% of valid pairs with one noise seed. With accuracies averaged over three seeds, it falls monotonically in 97%. The normalised drop differs by 0.048 between seeds on average (at most 0.121). There is no threshold below which removal is harmless (Fig. 1). This agrees with the trend that ViSoBERT [2] reports for fine-tuned encoders.

C. Teen code is light, not harmless

Teen code alone costs only 0.03–0.12 normalised drop per task (0.06 on average), against 0.33–0.47 for teen code followed by 50% stripping. Our teen-code table, however, changes just 6–12% of words. Partial diacritic removal at that same share of words does similar damage: 0.10 against 0.06 on VSFC and 0.11 against 0.12 on XNLI. The conclusion that teen code is harmless, here and in [3], should therefore be read as "teen code as usually generated changes few words".

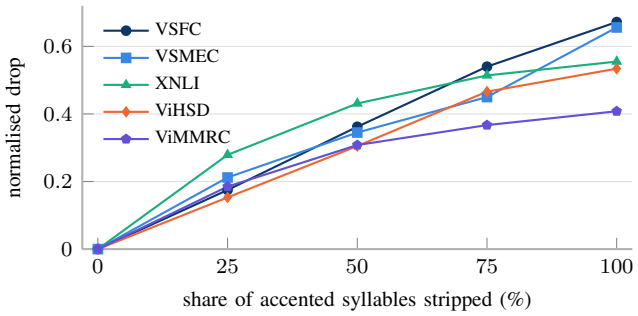


Fig. 1. Mean normalised drop over valid models up to 8B against the share of accented syllables stripped. Partial levels average three noise seeds; 100% is `strip`. Seed-averaged 95% CIs are about ±0.03–0.09 and are omitted for legibility.

TABLE IV
MEAN NORMALISED DROP UNDER `strip` FOR FOUR PROMPT TEMPLATES (85 PAIRS VALID UNDER ALL FOUR), AND SPEARMAN CORRELATION OF THE PER-PAIR DROPS WITH THOSE OF THE ORIGINAL TEMPLATE.

Template	d [95% CI]	Spearman
Original (Vietnamese)	0.52 [0.48, 0.55]	—
Vietnamese paraphrase	0.52 [0.49, 0.56]	0.96
English instructions	0.55 [0.51, 0.58]	0.95
Original + hint	0.52 [0.48, 0.56]	0.98

D. The loss does not depend on the prompt or the scoring

We re-ran the clean and stripped conditions with three more templates: a Vietnamese paraphrase of the instruction; English instructions with the Vietnamese input unchanged; and the original template plus a sentence warning that the text may lack diacritics ("hint"). Table IV reports the 85 (model, task) pairs, including the 14B models, that are valid under all four templates; this is why its mean drop (0.52) differs from the 0.57 of the 74 pairs up to 8B. The mean drop varies by at most 0.03, and models rank almost identically. Only one task–template combination differs significantly from the original prompt: XNLI with English instructions (+0.07).

The result is also stable across scoring choices. Over the 74 valid pairs of the models up to 8B (Table III), the mean drop under `strip` is 0.57 with calibration, 0.59 without, 0.71 in macro-F1, and 0.53 when the model's free-text answer is parsed instead of scored. Keeping the six pairs excluded by the gate changes little (ViHSD 0.53→0.59), and so does restricting the analysis to the 11 models valid on every task.

E. What the errors look like

Probability mass on the option letters barely changes under `strip`, so models still answer in the expected format: the errors are changes of judgement. Predictions do drift toward one label: under `strip`, the most frequent predicted label takes more than 60% of the predictions in 27 of 74 valid pairs (on VSFC, seven models drift toward *negative*). §VII-B tests whether re-estimating the calibration prior on the noisy inputs undoes this.

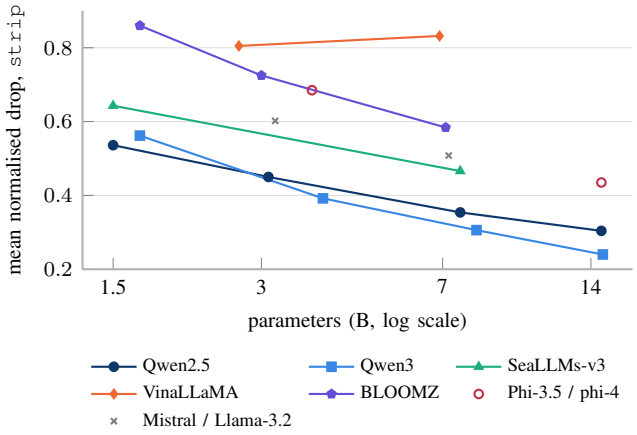


Fig. 2. Mean normalised drop under `strip` over each model's valid tasks against size (log scale); lines join sizes of one family. Models up to 8B from the main run, 14B models from the extension run.

V. Who Is Robust?

Size. We regress the `strip` drop of each valid pair on $\log_2$(parameters), with task and family fixed effects and standard errors clustered by model. Because there are only 11–19 clusters, Table V reports p-values from a t distribution with $G-1$ degrees of freedom and from a restricted wild cluster bootstrap (Webb weights, 9,999 draws), not from the normal approximation. The drop shrinks by 0.090 per doubling across the 16 models up to 8B, and by 0.086 $[-0.104, -0.068]$ when the three 14B models are added (wild bootstrap $p = 0.001$; 19 models, 89 pairs). With family fixed effects, families with a single model (Mistral, Llama-3.2, Phi-3.5, phi-4) do not inform the slope, so the Phi-3.5→phi-4 change of generation cannot drive it; dropping both Phi models or keeping only the three families with at least three sizes leaves the slope at -0.086 and -0.093. The 14B models lose 0.24 (Qwen3-14B), 0.30 (Qwen2.5-14B) and 0.44 (phi-4) on average. Size helps, but even the most robust 14B model still loses about a quarter of its above-chance accuracy. Pairwise tests within a family have less power: the largest model is significantly more robust than the smallest in only 5 of 22 family–task tests after Holm correction, four of them on VSFC (e.g. Qwen2.5, 0.73→0.27; Qwen3, 0.55→0.17).

Vietnamese adaptation: no evidence of more robustness. With size and task controlled, the four adapted models (SeaLLMs-v3 1.5B/7B, VinaLLaMA 2.7B/7B) lose 0.16 $[-0.04, 0.36]$ *more* than general models, which is not significant (wild bootstrap $p = 0.16$, Table V). This comparison mixes adaptation with the base model: SeaLLMs-v3 continues Qwen2, not the newer Qwen2.5 it shares a vocabulary with, and VinaLLaMA's Llama-2 base is not in our suite. We therefore read it as descriptive. SeaLLMs-v3 loses more than Qwen2.5 of the same size in 8 of 9 task comparisons (only VSFC survives Holm correction), and adapted models show a larger NLL increase under `strip` (2.96 vs. 1.71 nats per clean syllable); one possible reason is continued pre-training on edited, fully accented text, which we do not test here.

TABLE V
Inference with few clusters (clusters = models). Coefficient with a $t(G-1)$ 95% CI; p-values from the normal approximation, $t(G-1)$, and a restricted wild cluster bootstrap.

Sample	Coef. [95% CI]	$p_{\mathcal{N}}$	p_t	p_{wild}	G
Size: drop per doubling of parameters					
Up to 8B	-0.090 $[-0.11, -0.07]$	$6{\cdot}10^{-16}$	$8{\cdot}10^{-7}$	0.003	16
With 14B	-0.086 $[-0.10, -0.07]$	$1{\cdot}10^{-22}$	$1{\cdot}10^{-8}$	0.001	19
No Phi models	-0.086 $[-0.10, -0.07]$	$5{\cdot}10^{-23}$	$3{\cdot}10^{-8}$	0.001	17
$\geq$3 sizes only	-0.093 $[-0.12, -0.07]$	$3{\cdot}10^{-20}$	$3{\cdot}10^{-6}$	0.003	11
Adapted minus general (size and task controlled)					
Up to 8B	$+0.159$ $[-0.04, +0.36]$	0.086	0.11	0.16	16
With 14B	$+0.169$ $[-0.03, +0.37]$	0.072	0.089	0.14	19

TABLE VI
Linear probability models of flips: change in flip probability per SD of each predictor (percentage points, 95% CI). Each panel is a separate regression.

Predictor	Effect (pp/SD)	p
A. Same item: item×condition + model FE, all models up to 8B		
ΔNLL	+1.8 [1.0, 2.6]	2.5×10^{-5}
Δtokens	+2.6 [1.2, 4.0]	3.9×10^{-4}
Δbroken	-3.9 $[-5.6, -2.2]$	9.3×10^{-6}
A′. As A, without Phi-3.5 and Mistral		
ΔNLL	+1.9 [1.0, 2.9]	3.1×10^{-5}
Δbroken	-6.3 $[-9.3, -3.2]$	5.4×10^{-5}
B. Same item, Qwen2.5 / Qwen3 / SeaLLMs-v3 (one tokenizer)		
ΔNLL per clean syllable	+3.5 [2.4, 4.7]	3×10^{-9}
ΔNLL per token	+3.7 [2.5, 4.9]	2×10^{-9}
C. Model + task + condition FE, partial and strip only		
Ambiguity of stripped words	+2.0	1.6×10^{-5}
ΔNLL	+3.0	2.8×10^{-9}
Δtokens	-1.5	0.014

VI. What Predicts Errors?

For each noisy item that a model answered correctly when clean, we model whether it now errs (a *flip*) with a linear probability model. The main specification has item×condition and model fixed effects, so it compares different models on the same noisy item; standard errors are clustered by item. Its predictors vary across models: the standardised clean-to-noisy changes in

- NLL of the input fields per clean syllable (ΔNLL);
- tokens per syllable (Δtokens);
- share of syllables split across tokens (Δbroken).

The ambiguity of the stripped words (the number of their accented readings, in bits) is a property of the item, identical for every model, so the item×condition effects absorb it. We estimate it in a separate model with model, task and condition fixed effects instead (Table VI, panel C). All estimates are associations; none of them identifies a cause.

ΔNLL predicts errors. The model that finds a stripped item more surprising, relative to the clean version, is more likely to get it wrong (panel A). The association is twice as strong among Qwen2.5, Qwen3 and SeaLLMs-v3, whose

tokenization of each item is identical (panel B), so it does not come from tokenization differences, and it is the same whether NLL is normalised per clean syllable or per token. ΔNLL and flips may be two symptoms of one failure to reconstruct the meaning, so we do not read this as a mechanism. Across items, stripped words with more accented readings go with more flips (panel C).

Tokenization measures pull in opposite directions. Δtokens and Δbroken are strongly correlated ($r = 0.87$) yet enter with opposite signs, and the same pattern appears at the model level (Δfertility $+0.145$, $p = 0.019$; Δbroken -0.192, $p = 0.005$). Fragmentation therefore does not give a simple "more pieces, more errors" account. Phi-3.5 and Mistral even produce *fewer* tokens after stripping and still lose heavily.

An unexplained negative effect. More broken syllables after stripping go with *fewer* errors. We checked that this is not an artefact:

- *Collinearity*: the variance inflation factor is about 4, and the sign holds when Δbroken is entered alone (-1.6 pp, $p = 1.4 \times 10^{-4}$).
- *Clean tokenization*: the effect survives a control for the clean break rate (-3.8 pp, $p = 1.4 \times 10^{-5}$), which itself adds nothing ($p = 0.83$).
- *Subsamples*: it stays negative in all 14 leave-one-tokenizer-group and leave-one-family-out refits (-4.6 to -2.3 pp) and under another definition (-4.1 pp).
- *Direction of the tokenizer*: it is stronger, not weaker, when Phi-3.5 and Mistral, whose tokenizers split fewer syllables after stripping, are left out together (-6.3 pp; -2.3 ppentered alone; panel A′).

Tokenizers also differ in direction. Phi-3.5, Mistral and phi-4 split 80–90% of clean Vietnamese syllables and *fewer* after stripping (phi-4: 0.81→0.50), while all other tokenizers split more. We report this effect as open and make no causal claim about broken syllables.

VII. Can It Be Fixed?

A. A prompt hint does nothing

Adding the sentence "the text may be written without diacritics" changes clean and stripped accuracy by less than one point on every task. It recovers -1% of the lost accuracy on average, and no task's CI excludes zero in its favour.

B. Re-estimating the calibration prior helps little

Our main predictions divide by a prior estimated on clean dev items. A system that receives only unaccented input could instead estimate the prior on that input, without labels. Doing so on the stripped test inputs recovers 7% [5, 10] of the lost accuracy over the 74 valid pairs up to 8B (mean drop 0.57→0.52). It does reduce label collapse, from 27 to 14 pairs in which one label takes more than 60% of the predictions, but most errors are not a shift of the label prior.

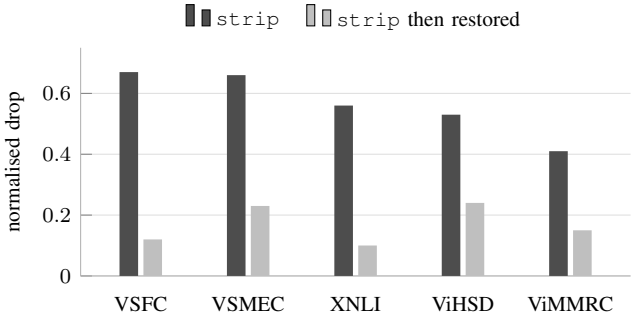


Fig. 3. Mean normalised drop over valid models up to 8B, before and after automatic diacritic restoration. A prompt hint leaves the `strip` bars unchanged (recovery -1%).

C. Restoring diacritics recovers most of the loss

We restore the stripped text with an off-the-shelf restorer, a token classifier fine-tuned from XLM-R large and released under Apache-2.0[1], using 120-word windows, and score the result as before. Restoration recovers 64% of the loss on average (65% on the same 74 pairs as §VII-B), and the remaining drop is 0.17 instead of 0.57 (Table III). On the 85 pairs of Table IV it recovers 60% (49–71% per task), while the hint recovers -1% (Fig. 3).

The benefit follows the restorer's accuracy. It is lower on the two social-media tasks, and not because their original text already lacks diacritics. Words that the original writes without diacritics where formal text uses them, together with slang and foreign words, make up only 4.5% (VSMEC) and 5.7% (ViHSD) of the words. Excluding them barely changes restoration accuracy (VSMEC 0.762→0.775, ViHSD 0.751→0.749). The restorer is simply weaker on informal text. Restoration also recovers less for the 14B models: 23% (Qwen2.5-14B), 42% (Qwen3-14B) and 50% (phi-4). These models lose less to begin with, so restoration errors weigh relatively more: after restoration, 14 of their 15 valid pairs remain significantly below clean accuracy (by 9, 5 and 7 points on average). The restorer was trained on Vietnamese news titles, a domain distinct from our test sets, so we do not expect memorised test items, although without its training data we cannot exclude overlap.

D. Naturally unaccented text

Synthetic stripping removes every diacritic from text that was written with them, whereas people who type without diacritics may write differently. After the same cleaning as our test items, 270 ViHSD test comments contain no diacritic at all. We keep the 226 in which the restorer changes at least 20% of the words (most of the other 44 are English, names or slang); on these, it adds diacritics to 70% of the words. Over 14 valid models, restoration changes balanced accuracy by +0.01 [-0.02, 0.04], whereas the same models gain +0.055 on synthetic ViHSD items. Only VinaLLaMA-7B gains significantly (+0.17), and no model loses significantly.

[1]https://huggingface.co/peterhung/vietnamese-accent-marker-xlm-roberta

Models do not read these comments well. Over the 14 models, mean balanced accuracy is 0.45 on the natural comments, the same as on our accented ViHSD test comments after stripping (0.45) and well below the same accented comments as written (0.56). On a scale where 0 is the stripped level and 1 the clean level, the median model sits at 0.12. The two sets contain different comments, so this is not a causal comparison, but it argues against the explanation that models already read habitual unaccented writing well. The remaining explanations are that the restorer fails on informal writing, or that these comments are harder for reasons unrelated to diacritics; a manually diacritized sample, in preparation, will separate them. The set is small and unbalanced (170 *clean*, 15 *offensive*, 41 *hate*), so per-model CIs are wide. We read this as a caution rather than a negative result: gains measured on synthetic stripping should not be assumed for real unaccented input.

VIII. Discussion

Three points matter for practitioners. First, unaccented input can cost a model about half of its above-chance accuracy, and prompt changes, including telling the model, do not fix it. Second, larger models lose less, steadily up to 14B, but not to zero. Third, restoring diacritics is cheap and helps on stripped formal text, but its benefit should be checked on the target users' own writing before relying on it.

For research, the main open question is the gap between synthetic and natural unaccented text. Closing it needs a larger, balanced labelled corpus of naturally unaccented text and a restorer trained on informal writing. A training-side remedy is a hypothesis worth testing rather than a conclusion of this study: if adding unaccented text to continued pre-training lowers ΔNLL, the association in §VI suggests, but does not establish, that the loss would shrink.

IX. Limitations

- *Scale*: the largest models have 14B parameters, and no closed model is tested; the size trend may flatten beyond 14B. Only two families (Qwen2.5, Qwen3) reach 14B; phi-4 differs from Phi-3.5 in generation and data as well as size, and with family fixed effects it does not inform the slope.
- *Restorer*: we use a single restorer, trained on Vietnamese news titles. Our test sets are not news titles, but overlap cannot be excluded without its training data. Its mean word accuracy on our data (83.4%) is below the 85% we set as a target in advance, so a better restorer may recover more.
- *Natural text*: the naturally unaccented set has 226 items from one task, selected with the restorer's output ($\geq$ 20% of words changed).
- *Adaptation*: our adapted models are not compared with their exact base models (Qwen2, Llama-2).
- *Synthetic noise*: real unaccented writing is not uniformly random, since people may drop diacritics on frequent words first.
- *Answer format*: we score single-letter answers. This measures understanding, not generation; free-generation agreement is gated, but generation tasks are not covered.
- *Teen code*: the table is small and hand-written.
- *Mechanism*: all item-level results are associations; ΔNLL and errors may share a cause, and the negative broken-syllable association is unexplained.

Ethics Statement

We use public research datasets under their licences. ViHSD contains offensive content; we show no examples from it. Derived noisy datasets will be released only where the original licences allow.

References

[1] J. Náplava, M. Straka, P. Straňák, and J. Hajič, "Diacritics restoration using neural networks," in *Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018)*, Miyazaki, Japan, 2018, pp. 1566–1573.

[2] N. Nguyen, T. Phan, D.-V. Nguyen, and K. Nguyen, "ViSoBERT: A pre-trained language model for Vietnamese social media text processing," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore, 2023, pp. 5191–5207.

[3] P. H. Ha, N. X. Phuc, N. N. Quy, T. T. Tran, N. M. Triet, and D. V. Thin, "How well do decision models transfer to Vietnamese? an empirical study of text classification," arXiv:2610.31422, 2026.

[4] D. V. Thin, D. N. Hao, and N. L.-T. Nguyen, "Prompt engineering with large language models for Vietnamese sentiment classification," in *Proceedings of the 38th Pacific Asia Conference on Language, Information and Computation (PACLIC 38)*, Tokyo, Japan, 2024, pp. 181–192.

[5] M. Tran, T. Chau, T.-N. Le, N. Tran, L. T. Nguyen, C. Dang, and D. Hoang, "How robust are LLMs to Vietnamese dialects?" arXiv:2608.10414, 2026.

[6] P. Rust, J. Pfeiffer, I. Vulić, S. Ruder, and I. Gurevych, "How good is your tokenizer? On the monolingual performance of multilingual language models," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL-IJCNLP)*, 2021, pp. 3118–3135.

[7] A. Petrov, E. La Malfa, P. H. S. Torr, and A. Bibi, "Language model tokenizers introduce unfairness between languages," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[8] K. Gorman and Y. Pinter, "Don't touch my diacritics," arXiv:2410.24140, 2024.

[9] P. Le-Hong, "Diacritics generation and application in hate speech detection on Vietnamese social networks," *Knowledge-Based Systems*, vol. 233, p. 107504, 2021.

[10] Y. Belinkov and Y. Bisk, "Synthetic and natural noise both break neural machine translation," in *International Conference on Learning Representations (ICLR)*, 2018.

[11] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond accuracy: Behavioral testing of NLP models with CheckList," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020, pp. 4902–4912.

[12] K. V. Nguyen, V. D. Nguyen, P. X. V. Nguyen, T. T. H. Truong, and N. L.-T. Nguyen, "UIT-VSFC: Vietnamese students' feedback corpus for sentiment analysis," in *2018 10th International Conference on Knowledge and Systems Engineering (KSE)*, 2018, pp. 19–24.

[13] V. A. Ho, D. H.-C. Nguyen, D. H. Nguyen, L. T.-V. Pham, D.-V. Nguyen, K. V. Nguyen, and N. L.-T. Nguyen, "Emotion recognition for Vietnamese social media text," in *Proceedings of PACLING 2019*, 2019.

[14] S. T. Luu, K. V. Nguyen, and N. L.-T. Nguyen, "A large-scale dataset for hate speech detection on Vietnamese social media texts," in *Advances and Trends in Artificial Intelligence (IEA/AIE 2021)*, 2021, pp. 415–426.

[15] A. Conneau, R. Rinott, G. Lample, A. Williams, S. R. Bowman, H. Schwenk, and V. Stoyanov, "XNLI: Evaluating cross-lingual sentence representations," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2018, pp. 2475–2485.

[16] K. V. Nguyen, K. V. Tran, S. T. Luu, A. G.-T. Nguyen, and N. L.-T. Nguyen, "Enhancing lexical-based approach with external knowledge for Vietnamese multiple-choice machine reading comprehension," *IEEE Access*, 2020.

[17] Q. Nguyen, H. Pham, and D. Dao, "VinaLLaMA: LLaMA-based Vietnamese foundation model," arXiv:2312.11011, 2023.

[18] H. Zhou, X. Wan, L. Proleev, D. Mincu, J. Chen, K. Heller, and S. Roy, "Batch calibration: Rethinking calibration for in-context learning and prompt engineering," in *International Conference on Learning Representations (ICLR)*, 2024.